BVS1 – Betriebssysteme

Lecture 03 – Preserving Execution and Building a Process · Wintersemester 2026/2027

Stavros Kousidis

TH Köln · ICCT

A New Requirement

Stop Here — Continue Later

Our current program simply runs.

Now imagine:

```
first_part();

process_yield();

second_part();
```

Required behaviour:

```
run first_part
→ kernel regains control
→ later resume
```

The Machine Already Knows About Continuations

Calls, `ra`, and Return

Conceptually:

```
first:
    call second
    # continue here later
```

A call needs:

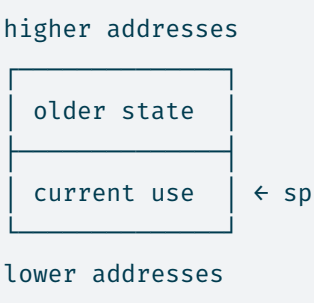
```
where to go
+
where to continue afterwards
```

RISC-V uses the return-address register:

```
ra
```

Calls Also Need a Stack

Nested calls need storage that survives while other functions run.



If execution A and execution B have different call histories:

*They need different saved **sp** values — and separate stack storage.*

Just Enough C

The C We Need for CP1

We will use familiar C ideas in a systems setting:

```
struct context c;         // grouped state
struct context *p = &c;   // address / pointer

uint8_t stack[4096];      // owned storage

void (*entry)(void);      // function to run
```

Read them as:

```
state object
address of state
stack storage
callable entry point
```

Save One Execution, Restore Another

The Calling Convention Lets Us Keep It Small

A function call already defines which register values must survive.

For our **cooperative** switch, a minimal context contains:

```
ra
sp
s0 ... s11
```

Why not save every register?

Because the switch happens at a normal function-call boundary.

We use the ABI instead of fighting it.

Represent the Context

```
struct context {
    uint64_t ra;
    uint64_t sp;
    uint64_t s[12];
};
```

This answers:

> *Where and with which preserved machine state can execution continue?*

It is still **not the whole process**.

Read the Tiny Assembly Boundary

Simplified:

```
switch_context:
    sd ra, 0(a0)
    sd sp, 8(a0)

    ld ra, 0(a1)
    ld sp, 8(a1)
    ret
```

```
a0 → context to save
a1 → context to restore
```

Trace the four memory operations.

The Final `ret` Is the Trick

After the loads:

```
ra = restored continuation
sp = restored stack position
```

Then:

```
ret
```

continues according to the **restored** `ra`.

It returns into the context we restored.

First Start and Later Resume

Kernel Context ↔ Process Context

For CP1:

```
kernel context
    ↕ switch_context
process context
```

When the process yields:

```
save process
restore kernel
```

When the kernel resumes it:

```
save kernel
restore process
```

Start Something That Never Ran

A paused process already has a saved continuation.

A **new** process does not.

So create one:

```
context.sp → top of new process stack
context.ra → process_bootstrap
```

The normal restore mechanism now doubles as the first-start mechanism.

The Bootstrap Returns Us to C

Conceptually:

```
static void process_bootstrap(void)
{
    current->entry();
    process_exit(0);
}
```

Assembly is needed only to cross the context boundary.

After that:

ordinary C becomes our main implementation language again.

Trace the Whole Control Transfer Once

```
kernel running
    ↓ switch
bootstrap on process stack
    ↓
process first_part
    ↓ yield
kernel resumes
    ↓ later switch
process_yield returns
    ↓
process second_part
```

At each arrow ask:

Which context is saved, and which is restored?

Now Call It a Process

Add Only What the Abstraction Needs

One useful teaching model:

```
struct process {
    struct context context;
    uint8_t stack[4096];
    void (*entry)(void);
    enum state state;
    int exit_status;
};
```

Context + owned stack + entry + lifecycle + completion.

Nothing more is required for CP1.

The Minimal Lifecycle

```
unused
   ↓ create
ready
   ↓ run
running
   ↓ yield
ready
   ↓ resume
running
   ↓ exit
finished
```

Three meanings:

Yield, Resume, Exit

Yield:

```
RUNNING → READY
save process → restore kernel
```

Resume:

```
READY → RUNNING
save kernel → restore process
```

Exit:

```
RUNNING → FINISHED
exit_status = 42
restore kernel
```

CP1 Has Now Been Enabled

The Complete Behaviour

CP1 — I have a process.

```
create
→ run on its own context / stack
→ yield
→ kernel regains control
→ resume
→ exit with status
→ kernel regains control
```

Every required concept in this sequence has now been introduced.

Lecture 4 will add **no new CP1 mechanism**.

Before You Leave

Can you explain:

1. why `ra` and `sp` matter?
2. why the process owns a separate stack?
3. how first start uses an artificial context?
4. what the switch assembly actually does?
5. why yield and exit are different?
6. why most process logic can now live in C?

Next:

Reconstruct and diagnose the process without following a recipe.